

Classes and object oriented programming in python3

Classes and Objects

- Object-Oriented Programming (OOP) organizes programs around objects that contain data and behavior.
- A class acts as a blueprint for creating objects.
- Objects are real instances created from a class.
- Classes help model real-world entities such as employees, dogs, products, and customers.
- OOP makes large applications easier to organize and maintain.
- Python treats everything as an object, including numbers, strings, lists, and functions.

Example

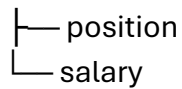
```
employee1 = Employee("Ji-Soo", 38, "developer", 1200)
employee2 = Employee("Lauren", 44, "tester", 1000)
```

Class Design

- A class defines attributes and methods that every object will have.
- Attributes store data about the object.
- Methods define actions the object can perform.
- Good class design groups related data and behavior together.
- Classes improve code readability and reusability.
- They help represent real-world concepts in software.

Example

```
Employee
├── name
└── age
```



Objects and Instance Attributes

- Each object has its own copy of attributes.
- Instance attributes store data unique to an object.
- Different objects can contain different values.
- Changes made to one object do not affect another object.
- Instance attributes are accessed using dot notation.
- They represent the state of an object.

Example

```
employee1.name = "Ji-Soo"  
employee2.name = "Lauren"
```

```
print(employee1.name)
```

Output:

Ji-Soo

Accessing Object Attributes

- Dot notation is used to access attributes.
- Attributes can be read or modified after object creation.
- Changes affect only the selected object.
- Attributes represent the current state of an object.
- Objects can store many related attributes.
- Dot notation is one of the most common OOP operations.

Example

```
print(employee1.name)
```

```
employee1.name = "Cynthia"
```

```
print(employee1.name)
```

Output:

Ji-Soo

Cynthia

Instance Methods

- Methods are functions defined inside a class.
- They describe the behavior of an object.
- Methods operate on object attributes.
- The first parameter is always self.
- Methods are called using dot notation.
- They help keep related functionality inside the class.

Example

```
employee2.increase_salary(20)
```

```
employee2.info()
```

Output:

Lauren is 44 years old.

Employee is a tester with salary \$1200

Class Introspection

- Python allows inspection of an object's class.
- Every object stores information about its class.

- `__class__` returns the class of an object.
- Introspection is useful for debugging.
- It helps understand object relationships.
- Many frameworks use introspection internally.

Example

```
print(employee2.__class__)
```

Output

```
<class '__main__.Employee'>
```

Real-World Class Modeling

- Classes are often modeled after real-world objects.
- Attributes describe characteristics.
- Methods describe actions.
- OOP makes software easier to understand.
- Real-world modeling improves maintainability.
- It creates natural relationships between objects.

Example

Dog

```
|— color  
|— age  
|— breed
```

Object:

Lassie

color = "White and Sable"

age = 2

breed = "rough collie"

Methods:

walking()
barking()
chase()

Object Instantiation

- Instantiation is the process of creating an object.
- Python automatically allocates memory for new objects.
- Constructors initialize object attributes.
- Multiple objects can be created from one class.
- Each object maintains its own state.
- Instantiation makes classes useful in programs.

Example

```
employee1 = Employee("Ji-Soo", 38, "developer", 1200)
employee2 = Employee("Lauren", 44, "tester", 1000)
```

Constructor Methods

- Python uses special methods called constructors.
- `__new__()` creates the object.
- `__init__()` initializes the object.
- Constructors run automatically during object creation.
- Initialization assigns values to attributes.
- Constructors ensure objects start in a valid state.

Example

```
class Employee:
```

```
    def __init__(self, name, age, position, salary):
        self.name = name
        self.age = age
```

```
self.position = position  
self.salary = salary
```

Dunder Methods

- Dunder stands for "double underscore."
- These are special methods with reserved behavior.
- Python automatically calls them in many situations.
- `__init__()` is the most commonly used dunder method.
- They support operator overloading and customization.
- Dunder methods make Python highly flexible.

Example

```
__init__()  
__new__()  
__str__()  
__repr__()
```

Abstraction

- Abstraction hides unnecessary implementation details.
- Users interact with the interface rather than internal logic.
- It reduces complexity.
- Only essential information is exposed.
- It improves usability and maintainability.
- Abstraction is a major pillar of OOP.

Example

```
employee.increase_salary(10)
```

The user does not need to know how the salary increase is calculated internally.

Encapsulation

- Encapsulation groups data and methods together.
- It protects internal object data.
- Objects control how data is accessed.
- It improves security and maintainability.
- Internal implementation details remain hidden.
- Encapsulation supports abstraction.

Example

class Employee:

```
def increase_salary(self):  
    pass
```

Users interact through methods instead of modifying internal logic directly.

Inheritance

- Inheritance allows one class to reuse another class's functionality.
- It follows the DRY (Don't Repeat Yourself) principle.
- Child classes inherit attributes and methods from parent classes.
- It reduces code duplication.
- It models "is-a" relationships.
- It improves maintainability and scalability.

Example

Employee is-a Person

Client is-a Person

Parent and Child Classes

- Parent classes contain shared functionality.
- Child classes extend parent functionality.
- Child classes inherit methods automatically.
- Additional attributes can be added in child classes.
- Code becomes more reusable.
- Class hierarchies become easier to manage.

Example

```
class Person:  
    pass
```

```
class Employee(Person):  
    pass
```

```
class Client(Person):  
    pass
```

Employee Hierarchy Example

Parent Class

```
Employee  
├── name  
├── age  
├── salary  
└── position
```

Child Classes

```
Developer(Employee)
```

```
Tester(Employee)
```

Developer:

```
framework  
increase_salary()
```

Tester:

run_tests()

Polymorphism

- Polymorphism allows different objects to respond differently to the same method.
- It provides flexibility in software design.
- Different child classes can implement behavior differently.
- The same interface works across multiple object types.
- It reduces conditional logic.
- It is one of the four OOP pillars.

Example

developer.increase_salary()

tester.increase_salary()

Both objects use the same method name but may behave differently.

OOP Pillars

- Abstraction hides complexity.
- Encapsulation protects data and behavior.
- Inheritance promotes code reuse.
- Polymorphism provides flexibility.
- Together they form the foundation of OOP.
- Most modern software systems rely heavily on these principles.

Example

Abstraction

Encapsulation

Inheritance

Polymorphism

Class Attributes and Methods

- Class attributes belong to the class itself.
- Instance attributes belong to individual objects.
- Class methods operate at the class level.
- Shared data can be stored in class attributes.
- They reduce duplication across objects.
- Python stores class metadata internally.

Example

Employee.__dict__

Output contains:

__init__

increase_salary

salary

Slots

- Slots reduce memory usage in objects.
- They prevent dynamic creation of attributes.
- Access to attributes becomes faster.
- Useful when creating many objects.
- They improve performance.
- Common in high-performance applications.

Example

class Employee:

__slots__ = ["name", "age", "salary"]

Data Classes

- Data classes automatically generate common methods.
- Less boilerplate code is required.
- They are ideal for storing structured data.
- Python provides them through the dataclasses module.
- They automatically create constructors.
- They improve readability and productivity.

Example

```
from dataclasses import dataclass
```

```
@dataclass
class Employee:
    name: str
    age: int
    salary: int
```

Why OOP Matters for Data Science

- Classes help represent datasets and business entities.
- OOP improves code organization in large projects.
- Machine learning libraries heavily use OOP concepts.
- Data pipelines become easier to manage.
- Reusable code reduces development time.
- Frameworks such as Pandas, Scikit-Learn, TensorFlow, and PyTorch are built around objects and classes.